Layered:

This pattern separates components into layers, where each layer only accesses the one below. This allows incremental development and makes outer layers easier to change, as only inner layers interact with hardware or the operating system (OS).

It's best for applications requiring a clear separation of concerns, high maintainability, and standardized development, such as traditional enterprise applications, CRUD websites, and desktop software

Some real world examples are:

- E-commerce sites:
  A presentation layer shows products, a business layer handles shopping cart logic, and a data layer manages inventory.

- Gmail:
  A classic example of a layered application for accessing and managing email.

Repository:

In this pattern all data is kept in a central repository and all components are separated – they can, however, interact through the repository. This is a particularly useful pattern if you have a lot of data.

It's best for separating domain logic from data access logic, providing a clean API for data retrieval and modification.

Some real world examples are:

- Finance Applications:
  A `TransactionRepository` could handle complex validation rules before storing data, ensuring business logic consistency.

- Git-based Collaboration:
  Platforms like Github or Gitlab allow teams to collaborate on complex projects, functioning as a shared digital workspace.

Client-server architecture:

In this pattern, a set of services is developed to run on servers. The clients then access the servers over a network to make use of the system. The network usage allows multiple clients to access the servers at one time to access the services the system provides.

This pattern is best for applications requiring centralized data management, security, and scalability, where a client (user device) requests services from a powerful server (system host). It is ideal for multi-user systems, such as web applications, secure banking systems, email systems, and cloud storage services.

Some real world examples are:

- Web Browsing:
  Your browser acts as the client sending HTTP requests to a web server to display websites.

- Gaming:
  Servers maintain the game state to ensure fair play and storage, ensuring access across multiple devices.

Pipe and filter architecture:

In this pattern, the data provided goes through a series of transformations (filters) as it flows from one component to another. This pattern is particularly useful for data processing.

This pattern is best for streaming data processing, complex transformations, and scenarios requiring high reusability and scalability. It breaks tasks into independent components (filters) linked by streams (pipes), enabling concurrent processing. It is ideal for ETL pipelines, compilers, and image processing.

Some real world examples are:

- Data Processing & ETL:
  Transforming raw data from diverse sources into usable formats for databases

- Image processing:
  Rendering pipelines, resizing, or watermarking

Combined Patterns:

1. Web-Based E-commerce Platform

Layered Architecture + Client-Server Architecture

In this setup, the overall system follows a Client-Server model where the user's browser (client) requests data from a central server over a network. Internally, that server is organized using a Layered architecture to separate the user interface logic from the core business rules and the database.

- Strengths:
    - The Layered pattern allows developers to update the "Presentation" layer (the website's look) without breaking the "Business" or "Database" layers.
    - The Client-Server model allows thousands of customers to access the system simultaneously via the network.
- Limitations:
    - If the central server or the network fails, no clients can access the services.
    - Data must travel through multiple layers and across a network, which can increase response times.

2. Big Data Analytics Dashboard:

Repository Architecture + Pipe and Filter Architecture

A data analytics system often uses a Repository to store massive amounts of raw information. To turn that raw data into useful charts, it uses a Pipe and Filter sequence to transform, clean, and calculate the data in stages.

- Strengths:
    - The Repository is ideal for managing the large volumes of data typical of "Big Data" systems.
    - New analytical "Filters" can be added easily to the data pipeline to create new types of reports without changing the storage layer.
- Limitations:
    - Managing the bidirectional flow of data between the central repository and various filters can become difficult to maintain.
    - Constant "parsing" and "unparsing" of data as it moves through various filters can lead to increased system computation time.
    - If the central Repository fails, the entire analytics pipeline loses its source of information and stops working.